

Reviewer Pack — GF(2)-Rank Defect of Term-Indicator Matrix Lower-Bounds Mono...

Entry 023dd98c00ee · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 00:11:13 UTC

GF(2)-Rank Defect of Term-Indicator Matrix Lower-Bounds Monotone DNF for k-CLIQUE

Entry ID: 023dd98c00ee **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 00:11:13 UTC

1. Conjecture

Field A (mathematical branch): Matroid theory — the GF(2) vector matroid M_F realized by the term-indicator matrix $A_F \in \mathbb{F}_2^{m \times N}$ of a monotone DNF $F = \bigvee_{i=1}^m T_i$ (rows $\{T_i\}$, columns indexed by variables). Its rank $\rho_2(F) := \text{rank}_{\mathbb{F}_2}(A_F)$ is the canonical computable matroid invariant in the Whitney-Welsh tradition; the corresponding ‘rank defect’ $\Delta(F) := \log_2(m(F)+1) - \log_2(\rho_2(F)+1)$ is a polymatroid-style quantity (subadditive log-rank vs. log-cardinality) that is rarely deployed in monotone circuit lower bounds (Razborov 1985 used Bollobás set-pair / sunflower closure, not GF(2) span of clutter indicators). **Field B** (complexity object): Monotone DNF representation size of the k-CLIQUE_n indicator on $N = C(n,2)$ edge variables; equivalently, the minimum-cardinality clutter of minimal terms whose union gives k-CLIQUE_n.

Statement:

Let F be a monotone DNF on N variables with minimal-term family $H_F = \{T_1, \dots, T_m\}$, A_F its $m \times N$ indicator matrix, and define $\Delta(F) := \log_2(m+1) - \log_2(\text{rank}_{\mathbb{F}_2}(A_F)+1)$. Conjecture: (a) $\Delta(F \wedge G) \leq \Delta(F) + \Delta(G) + 1$ and $\Delta(F \vee G) \leq \max(\Delta(F), \Delta(G)) + 1$ (approximate submodularity through Boolean operations); (b) $\Delta(F) \leq \log_2(N+1)$ for any monotone DNF with $m \leq \text{poly}(N)$ (since $\text{rank}_{\mathbb{F}_2}(A_F) \geq 1$ and $m \leq \text{poly}(N)$); (c) for the canonical k-CLIQUE_n DNF $F^{\wedge\{n,k\}}$ with $k = \lfloor n/2 \rfloor$, $\Delta(F^{\wedge\{n,k\}}) \geq n/2 - 3 \cdot \log_2 n - 4$ for all $n \geq 6$, witnessing a $\Omega(n)$ gap between log-cardinality and log-GF(2)-rank.

Rationale (proposer’s reasoning):

The GF(2) span of clique-edge-indicator vectors collapses to a sub-quadratic-rank object ($\text{rank} \leq C(n,2)$), while the minimal-term count $C(n, \lfloor n/2 \rfloor) \approx 2^{n/\sqrt{n}}$ is exponential — so the LOG ratio is $\Theta(n)$, much larger than any rank deficit attainable by poly-size clutters. Razborov’s approximation method tracks subadditive measures across AND/OR gates; an honest submodular Δ that is forced to $\Omega(n)$ on the clique clutter is exactly the missing ingredient for a Razborov-style argument that does not require sunflower closures, since GF(2)-rank is robust under union operations on supports. The bridge from matroid representability to monotone DNF size is concrete (one Gaussian elimination per gate), avoiding the natural-proofs / algebrization barriers that block spectral or Fourier approaches.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): af8a5d8c339698cb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds and $n \in \{6, 8, 10, 12\}$: (b) $\geq 99\%$ of 8100 random poly-DNFs satisfy $\Delta(F) \leq \log_2(N+1)$; (c) $\Delta(F^{\wedge \{n, \lfloor n/2 \rfloor\}}) \geq n/2 - 3\log_2 n - 4$ for all four n ; (a) $\geq 90\%$ of 30 random (F, G) pairs at $n=8$ obey $\Delta(F \wedge G) \leq \Delta(F) + \Delta(G) + 1$ and $\Delta(F \vee G) \leq \max(\Delta(F), \Delta(G)) + 1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - GF(2) rank monotone DNF k-clique lower bound - term indicator matrix matroid clutter monotone circuit complexity - log-rank cardinality polymatroid monotone formula clique

Top relevant hits considered: - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/2210.12983v2>] Poisson multi-Bernoulli mixture filter with general target-generated measurements and arbitrary clutter - [<http://arxiv.org/abs/2508.04609v2>] Analog Solver Circuit for Linear Symmetric Positive-Definite Systems at a Complexity Independent of Matrix Size - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/2309.13639v1>] A deletion-contraction formula and monotonicity properties for the polymatroid Tutte polynomial

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
```

```

rank = 0
for j in range(n):
    i_max = max(range(rank, m), key=lambda i: abs(A[i][j]))
    if A[i_max][j] == 0:
        continue
    A[rank], A[i_max] = A[i_max], A[rank]
    for i in range(m):
        if i != rank and A[i][j] != 0:
            factor = A[i][j] / A[rank][j]
            for k in range(n):
                A[i][k] -= factor * A[rank][k]
    rank += 1
return rank

def log2(x):
    return math.log2(x)

def canonical_clique_dnf(n, k):
    terms = []
    edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
    for clique in itertools.combinations(edges, k):
        term = [0] * len(edges)
        for edge in clique:
            term[edges.index(edge)] = 1
        terms.append(term)
    return terms

def random_poly_dnf(n, m, w):
    terms = []
    edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
    edge_set = set(random.sample(edges, min(m // w, len(edges))))
    for _ in range(m):
        term = [0] * len(edges)
        selected_edges = random.sample(edge_set, w)
        for edge in selected_edges:
            term[edges.index(edge)] = 1
        terms.append(term)
    return terms

def delta(F, N):
    m = len(F)
    rho_2 = gaussian_elimination(F)
    return log2(m + 1) - log2(rho_2 + 1)

n_values = [6, 8, 10, 12]
k_values = [3, 4, lambda n: n // 2]
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for k in k_values:
        if callable(k):
            k = k(n)
        F_clique = canonical_clique_dnf(n, k)
        F_random = random_poly_dnf(n, n, 2) + random_poly_dnf(n, n**2, 3) + random_poly_dnf(n,

```

```

for F in [F_clique] + F_random:
    instances_tested += 1
    delta_F = delta(F, n)

    if k == n // 2 and n >= 6:
        if delta_F < n / 2 - 3 * math.log2(n) - 4:
            conjecture_holds = False
            counterexample = f"n={n}, k={k},  $\Delta(F^{\{n,k\}})=\{\text{delta\_F}\}"

    if n == 8 and len(F_random) > 0:
        F1, F2 = random.sample(F_random, 2)
        delta_F1 = delta(F1, n)
        delta_F2 = delta(F2, n)
        delta_F_and = delta([t for t in F1 if any(t[i] == 1 for i in range(len(t)))],
                             delta_F_or = max(delta_F1, delta_F2)

        if delta_F_and > delta_F1 + delta_F2 + 1:
            conjecture_holds = False
            counterexample = f"n=8,  $\Delta(F \wedge G)=\{\text{delta\_F\_and}\}, \Delta(F)+\Delta(G)+1=\{\text{delta\_F1+delta\_F2}\}"

        if delta_F_or > max(delta_F1, delta_F2) + 1:
            conjecture_holds = False
            counterexample = f"n=8,  $\Delta(F \vee G)=\{\text{delta\_F\_or}\}, \max(\Delta(F), \Delta(G))+1=\{\max(\text{delta\_F1}, \text{delta\_F2})\}"

    return {
        "metric_name": "Delta",
        "metric_value": delta_F,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and sum(1 for r in results if not r["conjecture_holds"]) == 1:
        print(f"RESULT: FALSIFIED counterexample={next(r['counterexample'] for r in results if not r['conjecture_holds'])}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")$$$ 
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b7aab2a.py", line 118, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b7aab2a.py", line 78, in run_trial
    F_random = random_poly_dnf(n, n, 2) + random_poly_dnf(n, n**2, 3) + random_poly_dnf(n, n * math.floor(n/2), 3)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b7aab2a.py", line 56, in random_poly_dnf
    selected_edges = random.sample(edge_set, w)
                    ~~~~~
```

```
File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. ")
```

TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `random_poly_dnf` (`random.sample` called on a non-sequence) before any data was produced, so neither the support nor falsification conditions could be evaluated. | next: Fix `random_poly_dnf` by passing a list (e.g., `list(edge_set)` or `sorted(edge_set)`) to `random.sample` and rerun the 30-seed sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	337576
2	preregistration	claude_max	opus	0	0	6351
3	novelty	claude_max	opus	0	0	3239
4	novelty	claude_max	opus	0	0	9841
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14142
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15007
7	judge	claude_max	opus	0	0	3993

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 390150 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/023dd98c00ee.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/023dd98c00ee.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/023dd98c00ee.tar.gz` (if generated)